

```
// OSSP Skill 11 - Command History and Pipeline Structures
// Demonstrates history buffer, capacity, retrieval,
// pipeline command storage, execution order, and validation.
```

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
```

```
#define HISTORY_MAX 10
#define MAX_PIPE 8
```

```
static char *dup_string(const char *src) {
    size_t n = strlen(src) + 1;
    char *p = malloc(n);
    if (p) memcpy(p, src, n);
    return p;
}
```

```
typedef struct {
    char *items[HISTORY_MAX];
    int count;
} History;
```

```
typedef struct {
    char *commands[MAX_PIPE];
    int count;
} Pipeline;
```

```
static void add_history(History *h, const char *cmd) {
```



sai_pavani@Sai-Pavani: ~/OS_



GNU nano 8.7.1

skill_11.c.c

```
    if (h->count == HISTORY_MAX) {
        free(h->items[0]);
        memmove(&h->items[0], &h->items[1],
                (HISTORY_MAX - 1) * sizeof(char *));
        h->count--;
    }

    h->items[h->count++] = dup_string(cmd);
}

static void show_history(const History *h) {
    for (int i = 0; i < h->count; i++)
        printf("%2d  %s\n", i + 1, h->items[i]);
}

static void free_history(History *h) {
    for (int i = 0; i < h->count; i++)
        free(h->items[i]);
}

static void build_pipeline(Pipeline *p, const char *line) {
    char buffer[512];
    snprintf(buffer, sizeof(buffer), "%s", line);

    char *tok = strtok(buffer, "|");
    while (tok && p->count < MAX_PIPE) {
        while (*tok == ' ') tok++;

        p->commands[p->count++] = dup_string(tok);
    }
}
```

```
    tok = strtok(NULL, "|");
}
}

static void show_pipeline(const Pipeline *p) {
    printf("Pipeline contains %d command(s):\n", p->count);

    for (int i = 0; i < p->count; i++)
        printf("  Stage %d -> %s\n", i + 1, p->commands[i]);
}

static void free_pipeline(Pipeline *p) {
    for (int i = 0; i < p->count; i++)
        free(p->commands[i]);
}

int main(void) {
    History history = {0};
    char line[512];

    printf("=== Skill 11: History + Pipeline Structure ===\n");
    printf("Enter commands. 'history' displays history.\n");
    printf("A line containing | is displayed as a pipeline.\n");
    printf("Type 'exit' to quit.\n\n");

    while (1) {
        printf("history> ");
        if (!fgets(line, sizeof(line), stdin))
            break;
    }
}
```

```
    line[strcspn(line, "\n")] = '\0';

    if (strcmp(line, "exit") == 0)
        break;

    if (strcmp(line, "history") == 0) {
        show_history(&history);
        continue;
    }

    if (line[0] == '\0')
        continue;

    add_history(&history, line);

    if (strchr(line, '|')) {
        Pipeline p = {0};
        build_pipeline(&p, line);
        show_pipeline(&p);
        free_pipeline(&p);
    } else {
        printf("Stored command: %s\n", line);
    }
}

free_history(&history);
return 0;
}
```

```
sai_pavani@Sai-Pavani:~/OS_SKILL$ nano skill_11.c.c
sai_pavani@Sai-Pavani:~/OS_SKILL$ gcc skill_11.c.c -o skill_11
sai_pavani@Sai-Pavani:~/OS_SKILL$ ./skill_11
=== Skill 11: History + Pipeline Structure ===
Enter commands. 'history' displays history.
A line containing | is displayed as a pipeline.
Type 'exit' to quit.

history> ls
Stored command: ls
history> pwd
Stored command: pwd
history> date
Stored command: date
history> history
 1  ls
 2  pwd
 3  date
history> ls | wc -l
Pipeline contains 2 command(s):
  Stage 1 -> ls
  Stage 2 -> wc -l
history> cat file.txt | grep hello | wc -l
Pipeline contains 3 command(s):
  Stage 1 -> cat file.txt
  Stage 2 -> grep hello
  Stage 3 -> wc -l
history> exit
```